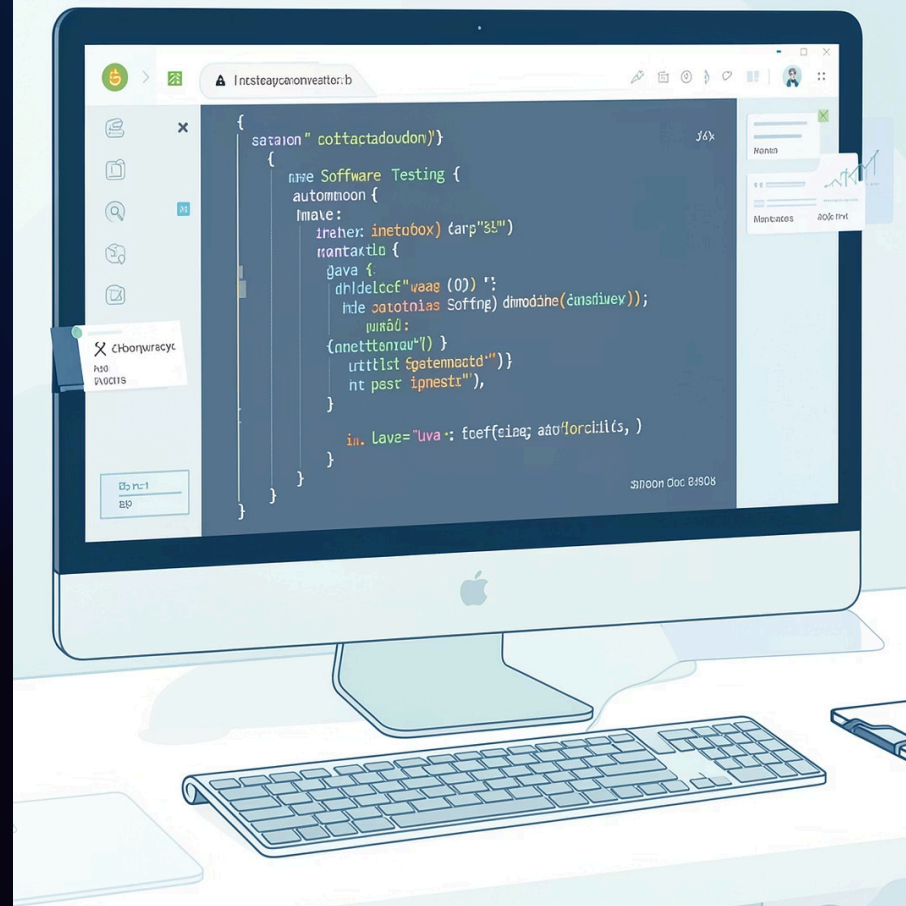


TestNG: Test Next Generation

Framework de Automação de Testes para Java — Versão 7.12.0

Fábio Terra e Felipe Barroso

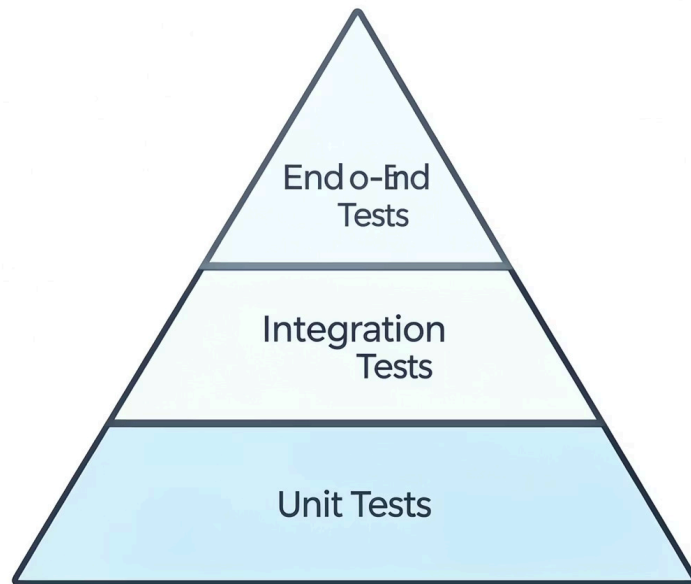


Origem e Evolução

O TestNG é um framework de automação de testes para o ecossistema Java, criado em 2004 por Cédric Beust para superar limitações arquiteturais do JUnit 3 e do NUnit. Sua proposta é uma abordagem mais flexível, poderosa e orientada a dados para o controle de qualidade.



Posicionamento na Pirâmide de Testes



O TestNG atua nos três níveis da pirâmide de testes, com destaque nos dois últimos:

- Unitário (base) – testes isolados de componentes
- Integração/API (meio) – validação de contratos e interfaces
- E2E/UI (topo) – fluxos completos do usuário

Seu sistema de agrupamento (smoke, regression, functional) ajuda a manter a proporção ideal entre camadas, evitando o antipadrão do "sorvete invertido".

Sistema de Anotações e Ciclo de Vida

O TestNG oferece um ciclo de vida hierárquico com anotações em múltiplos níveis de escopo:



Suíte XML

@BeforeSuite / @AfterSuite – Uma vez antes/depois de toda a suíte



Teste XML

@BeforeTest / @AfterTest – Antes/depois de cada tag



Classe

@BeforeClass / @AfterClass – Antes/depois de cada classe



Grupo

@BeforeGroups / @AfterGroups – Antes/depois de métodos de um grupo específico



Método

@BeforeMethod / @AfterMethod – Antes/depois de cada @Test



Lógica

@Test – Marca o caso de teste

Testes Orientados a Dados

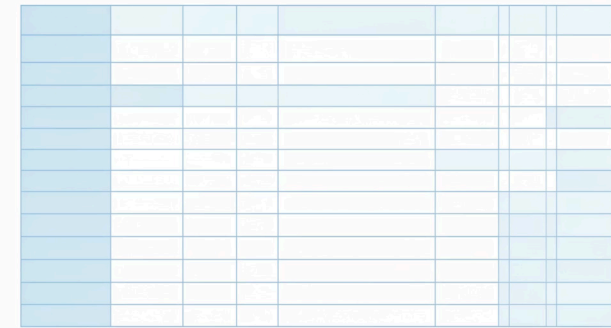
@DataProvider

Desacopla dados da lógica de teste — um método roda N vezes com N conjuntos de dados. Suporta fontes externas: Excel, CSV, arquivos .properties, banco de dados SQL.

```
@DataProvider(parallel = true)
public Object[][] credentials() {
    return new Object[][] {
        {"user1", "pass1"},
        {"user2", "pass2"},
        {"invalid", "wrong"}
    };
}
```

@Parameters

Parametrização estática via testng.xml (ex: URL de ambiente, nome do browser) sem recompilar o código.





Paralelismo

parallel=true no @DataProvider
ativa threads simultâneas por linha
de dados

Execução Paralela e Integrações



Paralelismo Nativo

Configurável via testng.xml:
parallel="methods|classes|tests" + thread-count. Não requer extensões externas.



Dependências

dependsOnMethods /
dependsOnGroups – cria
cadeias lógicas de
execução. Teste bloqueado
por falha é marcado como
Skipped.



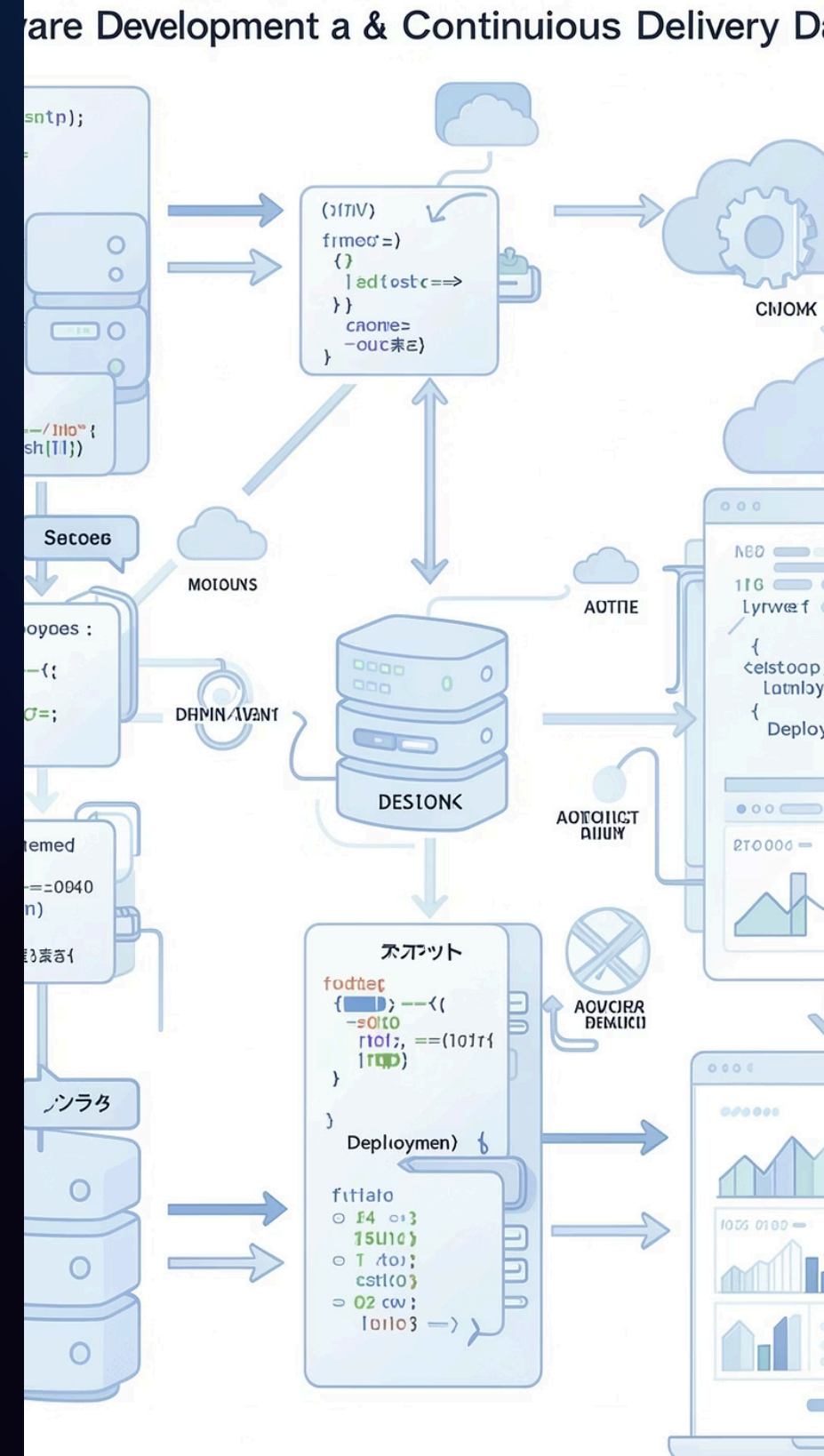
Ecosistema

Selenium · Appium · JaCoCo
· Mockito · Maven Surefire ·
Gradle · Jenkins · GitLab CI ·
GitHub Actions



IDEs

Eclipse · IntelliJ · NetBeans
– Suporte completo nas
principais ferramentas de
desenvolvimento



Tipos de Teste

Caixa-Branca (White-Box)

- Cobertura de declaração e ramificação
- Verificação de algoritmos e regras de negócio internas
- Segurança no nível do código
- Integração com JaCoCo (cobertura) e Mockito (isolamento)

Caixa-Preta (Black-Box)

- Testes de UI com Selenium (fluxos completos)
- Testes de integração de API (validação de contratos)
- Testes de Aceitação do Usuário (UAT)
- Partição de Equivalência e Análise de Valor Limite

Comparativo de Performance

Benchmark 2025 mostra o TestNG otimizado para paralelismo e gerenciamento robusto de dependências:

Métrica	JUnit 5	TestNG
Utilização de CPU	85%	90%
Tempo médio de espera de thread	120ms	80ms
Uso de memória de pico	850 MB	900 MB
Paralelismo nativo	Parcial (via extensões)	✓ Nativo via XML
Gerenciamento de dependências	Limitado	✓ Robusto
Relatórios nativos	Básicos	✓ HTML + XML

Fonte: Benchmark 2025 – Testes executados em ambiente controlado com 1000 casos de teste

Casos de Sucesso Global

Mais de 18.000 empresas utilizam o TestNG globalmente em sistemas mission-critical:

18K+

Empresas Globais

Utilizando TestNG em produção

4.2K+

Clientes EUA

Maior concentração de usuários

1K+

Índia, UK, Canadá

Expansão regional consolidada

Destaque em Setores:

- **Finanças:** Visa, HSBC, Oracle
- **Consultoria:** Capgemini, Accenture
- **Enterprise:** Sistemas de pagamento e plataformas corporativas





Quando Adotar o TestNG

✓ Adotar

- Projetos enterprise com suítes complexas de UI/API
- Necessidade de paralelismo real configurável
- Múltiplos ambientes e estratégias de regressão sofisticadas
- Orquestração de qualidade em sistemas Java de grande escala

⚠ Considerar Alternativas

- Microserviços simples com poucos testes
- Equipes iniciantes em automação
- Projetos onde JUnit/Spring Boot já consolidado

Conclusão: O TestNG é a escolha definitiva para orquestração de qualidade em sistemas Java de grande escala — superior ao JUnit 5 onde paralelismo, dependências e integração entre componentes são prioritários.